

REACT COMPLETE NOTES

WHAT IS REACT?

React is an open-source JavaScript library used for building fast, interactive, and reusable user interfaces (UI), especially Single Page Applications (SPA).

- Developed by: Meta (Facebook)
- Type: Frontend JavaScript Library
- Based on Components
- Uses Virtual DOM for faster updates
- Follows One-Way Data Flow (Parent → Child)

REACT PROJECT STRUCTURE

my-app/

├─ public/

├─ src/

| ├─ assets/

| ├─ components/

| ├─ pages/

| └─ App.js

| └─ index.js

└─ package.json

VIRTUAL DOM

Virtual DOM is a lightweight copy of the Real DOM.

When State or Props change:

1. React creates a new Virtual DOM.
2. Compares it with the previous Virtual DOM (Diffing).
3. Updates only the changed part in the Real DOM.

Result:

✓ Faster Rendering

✓ Better Performance

=====

□ COMPONENTS

Components are reusable building blocks of React applications.

Functional Component

```
function Hello() {  
  return <h1>Hello World</h1>;  
}
```

Arrow Function Component

```
const Hello = () => {  
  return <h1>Hello World</h1>;  
}
```

```
};
```

Rules:

- Component name should start with Capital Letter.
- Every component returns JSX.

=====

JSX (JavaScript XML)

JSX allows us to write HTML inside JavaScript.

Example

```
const name = "Ali";
```

```
return <h1>Hello {name}</h1>;
```

JSX Rules

- Return only one parent element.
- Use className instead of class.
- JavaScript expressions are written inside {}.
- Every tag must be properly closed.

=====

PROPS

Props (Properties) are used to pass data from Parent Component to Child Component.

Example

```
function Student(props) {  
  return <h1>{props.name}</h1>;  
}
```

```
<Student name="Ali" />
```

Characteristics

- ✓ Read Only
- ✓ Parent → Child
- ✓ Used for communication between components

=====

□ STATE (useState Hook)

State stores data that can change during the execution of the application.

The useState Hook allows us to update the state at runtime.

Whenever state changes, React automatically re-renders the component and updates the UI.

Syntax

```
const [state, setState] = useState(initialValue);
```

Example

```
import { useState } from "react";

function Counter() {

  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count : {count}
    </button>
  );
}
```

Explanation

count → Current State

setCount() → Updates State

0 → Initial Value

Never Do This

```
count = count + 1;
```

Always Do This

```
setCount(count + 1);
```

If new value depends on previous value

```
setCount(prev => prev + 1);
```

=====

HOOKS

Hooks are special functions that allow Functional Components to use React features.

Common Hooks

✓ useState

✓ useEffect

✓ useContext

✓ useRef

✓ useMemo

✓ useCallback

✓ useReducer

Rules of Hooks

- Always call Hooks at the top level.
- Never call Hooks inside loops or conditions.
- Hooks always start with "use".

=====

useEffect Hook

Used for Side Effects.

Examples

✓ API Calling

✓ Timers

✓ Event Listeners

✓ Logging

Syntax

```
useEffect(() => {
```

```
  console.log("Mounted");
```

```
}, []);
```

Runs Only Once

```
useEffect(() => {
```

```
}, []);
```

Runs Every Render

```
useEffect(() => {
```

```
});
```

Runs When Dependency Changes

```
useEffect(() => {
```

```
}, [count]);
```

Cleanup Function

```
useEffect(() => {
```

```
  return () => {
```

```
    console.log("Component Unmounted");
```

```
  };
```

```
}, []);
```

=====

EVENT HANDLING

```
function App() {
```

```
  const handleClick = () => {
```

```
    alert("Button Clicked");
```

```
  };
```

```
  return (
```

```
<button onClick={handleClick}>
  Click Me
</button>

);

}
```

=====

CONDITIONAL RENDERING

Using Ternary Operator

```
{
  isLoggedIn
    ? <h1>Welcome</h1>
    : <h1>Please Login</h1>
}
```

Using && Operator

```
{
  isAdmin && <h1>Admin Panel</h1>
}
```

=====

LISTS & KEYS

```
const names = ["Ali", "John", "Sara"];
```

```
names.map((name, index) => (  
  <li key={index}>{name}</li>  
));
```

Best Practice

Use unique id instead of index whenever possible.

=====

FORMS (Controlled Components)

```
import { useState } from "react";
```

```
function Form() {
```

```
  const [name, setName] = useState("");
```

```
  return (  
    <input  
      value={name}  
      onChange={(e) => setName(e.target.value)}  
    />  
  );  
}
```

Controlled Component

✓ React controls input using State.

=====

REACT ROUTER

Install

```
npm install react-router-dom
```

Example

```
import {  
  BrowserRouter,  
  Routes,  
  Route  
} from "react-router-dom";
```

```
function App() {
```

```
  return (  
  

```

```
    <BrowserRouter>
```

```
      <Routes>
```

```
<Route path="/" element={<Home />} />
```

```
<Route path="/about" element={<About />} />
```

```
</Routes>
```

```
</BrowserRouter>
```

```
);
```

```
}
```

=====

API CALLING

Using Fetch

```
useEffect(() => {
```

```
  fetch("https://api.example.com/data")
```

```
    .then(res => res.json())
```

```
    .then(data => console.log(data));
```

```
}, []);
```

=====

STYLING IN REACT

1. Inline CSS

```
<h1 style={{color:"red"}}>Hello</h1>
```

2. External CSS

```
import './App.css';
```

3. CSS Modules

```
import styles from './App.module.css';
```

□ FRAGMENTS

Used to avoid unnecessary div elements.

```
<>
```

```
<h1>Hello</h1>
```

```
<p>World</p>
```

```
</>
```

🔄 REACT RENDERING FLOW

User Action

|



Event Handler

|



setState()

|



State Updated

|



Virtual DOM Created

|



Diffing Algorithm

|



Real DOM Updated

|



UI Updated

IMPORTANT REACT RULES

- ✓ Component names start with Capital Letter.
- ✓ Hooks always start with "use".
- ✓ Never modify State directly.
- ✓ Always use setState() function.
- ✓ Every list item should have a unique key.
- ✓ Props are Read Only.
- ✓ State is mutable using Setter Function.
- ✓ Data flows only from Parent → Child.

=====

INTERVIEW POINTS

React → JavaScript Library

JSX → HTML inside JavaScript

Component → Reusable UI Block

Props → Parent to Child Data

State → Dynamic Data inside Component

useState → Manage State

useEffect → Side Effects

Virtual DOM → Faster Rendering

React Router → Navigation

Key → Unique Identity for List Items

=====

FINAL SUMMARY

- ✓ React is Component Based.
- ✓ Components are Reusable.
- ✓ JSX allows HTML inside JavaScript.
- ✓ Props pass data from Parent to Child.
- ✓ State stores dynamic data.
- ✓ useState updates state at runtime.
- ✓ React automatically re-renders when state changes.
- ✓ useEffect handles side effects.
- ✓ Virtual DOM improves performance.
- ✓ React Router manages navigation.
- ✓ React follows One-Way Data Flow.

=====

END OF NOTES